

SPARQL: SPARQL Protocol And RDF Query Language

**Ambient intelligence course (A.Y. 2025/2026)
Topic taught by Prof. Floriano Scioscia**

Gabriele Colapinto

May 31, 2026

License

Notes from Ambient intelligence (A.Y. 2025/2026) by Gabriele Colapinto. Course taught by professors Michele Ruta, Floriano Scioscia, Arnaldo Tomasino, Davide Loconte — Licensed under CC BY-NC 4.0.

<https://creativecommons.org/licenses/by-nc/4.0/>

Contents

1	SPARQL Protocol and RDF Query Language	5
1.1	Role of SPARQL in RDF Data Access	5
1.2	Triple Patterns and Solution Bindings	5
1.2.1	Triple Patterns	5
1.2.2	Example RDF Graph	6
1.2.3	A Basic SELECT Query	6
1.3	Result Sets and XML Serialization	7
1.4	Structure of a SPARQL Query	8
1.5	IRIs, Prefixes, and Dataset Definitions	8
1.5.1	Prefixed Names and Relative IRIs	8
1.5.2	Default and Named Graphs	10
1.6	Graph Patterns in the WHERE Clause	14
1.6.1	Basic Graph Patterns	14
1.6.2	OPTIONAL Patterns	15
1.6.3	UNION Patterns	15
1.6.4	FILTER Conditions	16
1.6.5	GRAPH Patterns	16
1.7	Query Modifiers	17
1.7.1	Ordering	17
1.7.2	Limiting and Offsetting Results	17
1.8	Selected Operators and Functions	18
1.9	Query Forms	18
1.9.1	SELECT	18
1.9.2	CONSTRUCT	20
1.9.3	ASK	21
1.9.4	DESCRIBE	21
1.10	Scope of the Query Language	21

1 SPARQL Protocol and RDF Query Language

1.1 Role of SPARQL in RDF Data Access

SPARQL, the SPARQL Protocol and RDF Query Language, is the standard query language for RDF datasets. Since an RDF dataset is naturally represented as a directed labelled graph made of triples, the central operation in SPARQL is **graph pattern matching**: a query describes a pattern, and the query processor searches the RDF data for substitutions of variables that make the pattern match.

A SPARQL query is not, by itself, an inference procedure. A query processor evaluates graph patterns over the RDF dataset exposed by the endpoint, possibly according to the entailment regime supported by that endpoint, but the basic language should not be confused with a rule engine or an ontology reasoner. Its primary role is to retrieve, test, or construct information from RDF graphs.

A **SPARQL endpoint** is a software service that accepts SPARQL queries and returns query results, commonly over HTTP. Results may be returned in machine-processable formats such as XML, JSON, or RDF serializations, or in human-readable formats such as HTML. Endpoints can be classified according to the datasets they expose:

- a **specific endpoint** provides access to a predefined set of loaded datasets, such as <https://dbpedia.org/sparql>;
- a **generic endpoint** allows the query to identify the Web-accessible RDF dataset to be queried, as in <https://www.openlinksw.com/sparql>.

The availability of public RDF datasets and endpoints is one of the practical reasons for SPARQL's importance in Linked Data applications: large knowledge graphs can be queried through a uniform language without requiring a proprietary data access interface. Collections such as <https://lod-cloud.net> provide entry points for discovering datasets and endpoints in the Linked Open Data cloud.

1.2 Triple Patterns and Solution Bindings

1.2.1 Triple Patterns

SPARQL queries are built from **triple patterns**. A triple pattern has the same shape as an RDF triple, but any of the three positions—subject, predicate, or object—may contain a variable. Variables are prefixed by `?` and act as unknowns to be bound during pattern matching.

The most general triple pattern is:

`?subject ?predicate ?object`

A more selective pattern fixes the predicate and leaves the subject and object unknown:

`?movie dbpedia-owl:starring ?actor`

This pattern matches triples whose predicate is `dbpedia-owl:starring`. For every matching triple, the subject is bound to `?movie` and the object is bound to `?actor`. The result of evaluating a graph pattern is therefore a set of **solution mappings**, where each solution maps query variables to RDF terms.

1.2.2 Example RDF Graph

Consider the following RDF graph, written in Turtle syntax. It states the actors starring in two movies.

```
@prefix dbpedia: <http://dbpedia.org/resource/> .
@prefix dbpedia-owl: <http://dbpedia.org/ontology/> .

dbpedia:The_Matrix_Revolutions dbpedia-owl:starring
    dbpedia:Carrie-Anne_Moss ,
    dbpedia:Keanu_Reeves ,
    dbpedia:Laurence_Fishburne .

dbpedia:Speed dbpedia-owl:starring
    dbpedia:Keanu_Reeves ,
    dbpedia:Sandra_Bullock .
```

Although Turtle abbreviates repeated subjects and predicates, the graph contains five RDF triples. Each actor listed after `dbpedia-owl:starring` corresponds to one triple.

1.2.3 A Basic SELECT Query

The following query retrieves movies and actors by matching the pattern `?movie dbpedia-owl:starring ?actor` against the RDF graph.

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>

SELECT ?movie ?actor
FROM <http://dbpedia.org>
WHERE {
    ?movie dbpedia-owl:starring ?actor
}
```

The **PREFIX** declaration abbreviates IRIs. The **SELECT** clause specifies the variables to be returned. The **FROM** clause identifies the default graph to be queried, and the **WHERE** clause contains the graph pattern.

For the example graph, all five triples match the pattern. The result set is a table of bindings:

?movie	?actor
dbpedia:The_Matrix_Revolutions	dbpedia:Carrie-Anne_Moss
dbpedia:The_Matrix_Revolutions	dbpedia:Keanu_Reeves
dbpedia:The_Matrix_Revolutions	dbpedia:Laurence_Fishburne
dbpedia:Speed	dbpedia:Keanu_Reeves
dbpedia:Speed	dbpedia:Sandra_Bullock

This result resembles a relational projection: only the variables named in the **SELECT** clause appear in the result. However, the underlying evaluation is graph-pattern matching over RDF terms, not tuple matching over fixed relational schemas.

1.3 Result Sets and XML Serialization

A **SELECT** query returns a solution sequence: each solution contains bindings for some or all projected variables. One standardized serialization of such results is the SPARQL Query Results XML format.

The following fragment illustrates the structure of an XML result document for a query that projects **?actor**.

```
<sparql xmlns="http://www.w3.org/2005/sparql-results#">
  <head>
    <!-- one variable element for each query variable -->
    <variable name="actor"/>
  </head>

  <results distinct="false" ordered="true">

    <!-- one result element for each query result -->
    <result>
      <binding name="actor">
        <uri>http://dbpedia.org/resource/Keanu_Reeves</uri>
      </binding>
    </result>

    <result>
      <binding name="actor">
        <uri>http://dbpedia.org/resource/Carrie-Anne_Moss</uri>
      </binding>
    </result>

  </results>
</sparql>
```

The root element is **sparql**. The **head** element lists the variables projected by the query. The **results** element contains one **result** element for each solution. A **binding** associates a variable name with an RDF term, represented for example as a **uri**, a literal, or a blank node. For boolean queries, the XML result uses a **boolean** element instead of a list of variable bindings.

The attributes **distinct** and **ordered** describe whether duplicate solutions have been removed and whether the solution sequence is ordered. These properties depend on query modifiers such as **DISTINCT** and **ORDER BY**.

1.4 Structure of a SPARQL Query

A SPARQL query is organized into a sequence of clauses. Not all clauses are mandatory, but the following structure captures the main components used in typical queries.

```
# Prefix declarations
PREFIX ...
BASE ...

# Query form
SELECT ...
CONSTRUCT ...
ASK ...
DESCRIBE ...

# Dataset definition
FROM ...

# Query patterns
WHERE { ... }

# Modifiers
ORDER BY ...
LIMIT ...
OFFSET ...
```

The **prefix declarations** define abbreviations for IRIs. The **query form** determines the kind of result produced by the query: a table of bindings, an RDF graph, a boolean value, or a description of resources. The **dataset definition** identifies the RDF graph or graphs to be queried. The **query pattern** is evaluated against the dataset. Finally, **modifiers** control ordering, duplicate handling, and pagination of the result sequence.

1.5 IRIs, Prefixes, and Dataset Definitions

1.5.1 Prefixed Names and Relative IRIs

SPARQL supports two mechanisms for shortening IRIs: **PREFIX** and **BASE**.

A **PREFIX** declaration associates a prefix name with an IRI. A prefixed name is then formed by joining the prefix with a local part.

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
```


The prefixed name `dbpedia:The_Matrix` denotes the IRI obtained by concatenating the prefix IRI with the local part `The_Matrix`.

A **BASE** declaration defines a base IRI used to resolve relative IRIs.

```
BASE <http://dbpedia.org/>
PREFIX dbpedia: <http://dbpedia.org/resource/>

# The following forms denote the same IRI:
<http://dbpedia.org/resource/The_Matrix> # Full IRI
dbpedia:The_Matrix                       # Prefixed IRI
<resource/The_Matrix>                   # Relative IRI
```

The first form is an absolute IRI. The second is a prefixed name. The third is a relative IRI resolved against the base IRI. In practice, prefixes are the most common abbreviation mechanism in SPARQL queries because they also make the vocabulary used by the query explicit.

Current Base IRI and Relative IRI Resolution

A **BASE** declaration does not introduce a named base that can later be selected explicitly. Instead, it sets the *current base IRI* used to resolve relative IRI references occurring in the query. At any point in the query text, a relative IRI is interpreted with respect to the base IRI currently in force at that position.

This means that the absence of a local disambiguation mechanism does not imply that a SPARQL query may contain only one **BASE** declaration. Multiple **BASE** declarations may occur, but they act sequentially: a later declaration changes the current base for the relative IRIs that follow it.

```
BASE <http://example.org/a/>

SELECT *
WHERE {
  <resource/x> ?p ?o .
}
```

In this query, the relative IRI `<resource/x>` is resolved as:

```
http://example.org/a/resource/x
```

If a second **BASE** declaration occurs later, subsequent relative IRIs are resolved against the new base:

```
BASE <http://example.org/a/>
PREFIX ex: <vocab/>

BASE <http://example.org/b/>
SELECT *
WHERE {
  <resource/x> ?p ?o .
```

```
}
```

Here, the relative IRI in the **PREFIX** declaration is resolved using the first base, so **ex:** denotes:

```
http://example.org/a/vocab/
```

The relative IRI **<resource/x>** in the query body, instead, is resolved using the second base:

```
http://example.org/b/resource/x
```

The disambiguation is therefore positional rather than syntactic: SPARQL does not provide a construct to choose among several bases at the point where a relative IRI is written. A relative IRI is always resolved against the current base IRI. When different resolution contexts are needed, the query should either change the current base with another **BASE** declaration, use absolute IRIs, or define suitable **PREFIX** declarations.

1.5.2 Default and Named Graphs

A SPARQL query is evaluated over an *RDF dataset*. An RDF dataset contains exactly one default graph and zero or more named graphs. The default graph has no name and is used for matching ordinary graph patterns. Named graphs, instead, are identified by IRIs and are accessed explicitly with the **GRAPH** keyword.

The graph currently used to evaluate a graph pattern is called the *active graph*. Outside a **GRAPH** block, the active graph is the default graph. Inside a **GRAPH** block, the active graph is the named graph selected by that block.

No Explicit Dataset Definition

If a query contains neither **FROM** nor **FROM NAMED**, the dataset is not specified inside the query itself. In that case, the query is evaluated over the dataset provided by the SPARQL service or by the protocol request.

```
SELECT ?s ?p ?o
WHERE {
  ?s ?p ?o .
}
```

In this case, the triple pattern is matched against the service-defined default graph. Whether named graphs are available depends on the dataset exposed by the endpoint. Therefore, the meaning of the query is endpoint-dependent.

A Single FROM Clause

The **FROM** clause specifies a graph that contributes to the default graph of the query dataset.

```
SELECT ?s ?p ?o
FROM <http://example.org/graph1>
WHERE {
```

```
?s ?p ?o .
}
```

The pattern inside **WHERE** does not use **GRAPH**, so it is matched against the default graph. In this query, the default graph is obtained from `<http://example.org/graph1>`.

It is important to distinguish the graph used to build the default graph from a named graph. A graph mentioned with **FROM** is not automatically available as a named graph. Therefore, the following query does not necessarily match anything, even though the same IRI appears in **FROM**:

```
SELECT ?s ?p ?o
FROM <http://example.org/graph1>
WHERE {
  GRAPH <http://example.org/graph1> {
    ?s ?p ?o .
  }
}
```

The **GRAPH** keyword searches among the named graphs of the query dataset. Since **FROM** contributes to the default graph and does not declare a named graph, there is no named graph to select unless it is also declared with **FROM NAMED** or provided by the dataset definition.

Multiple **FROM** Clauses

A query may contain more than one **FROM** clause. In that case, the default graph is the RDF merge of the graphs identified by the **FROM** clauses.

```
SELECT ?s ?p ?o
FROM <http://example.org/graph1>
FROM <http://example.org/graph2>
WHERE {
  ?s ?p ?o .
}
```

The query pattern is evaluated as if **graph1** and **graph2** had been combined into a single default graph for the purpose of query evaluation. This is useful when data from several sources should be queried without preserving the distinction between their graph names.

The merge affects only the default graph. The original graph IRIs are not available through **GRAPH** merely because they occur in **FROM**. Consequently, the query cannot recover from which **FROM** graph a matched triple originated, unless such provenance information is explicitly represented in the data itself.

A Single **FROM NAMED** Clause

The **FROM NAMED** clause declares a named graph in the query dataset. Unlike **FROM**, it does not add the graph to the default graph.

```
SELECT ?s ?p ?o
FROM NAMED <http://example.org/graph1>
```

```
WHERE {  
  GRAPH <http://example.org/graph1> {  
    ?s ?p ?o .  
  }  
}
```

Here, the triple pattern is evaluated against the named graph `<http://example.org/graph1>` because the **GRAPH** block explicitly selects that graph.

If a query contains **FROM NAMED** clauses but no **FROM** clauses, the default graph is empty. Therefore, ordinary triple patterns outside **GRAPH** do not match triples from the named graph:

```
SELECT ?s ?p ?o  
FROM NAMED <http://example.org/graph1>  
WHERE {  
  ?s ?p ?o .  
}
```

This query attempts to match `?s ?p ?o` against the default graph, not against `graph1`. Since `graph1` is available only as a named graph, it must be accessed using **GRAPH**.

Multiple **FROM NAMED** Clauses

A query may declare several named graphs.

```
SELECT ?g ?s ?p ?o  
FROM NAMED <http://example.org/graph1>  
FROM NAMED <http://example.org/graph2>  
WHERE {  
  GRAPH ?g {  
    ?s ?p ?o .  
  }  
}
```

When the argument of **GRAPH** is a variable, SPARQL tries the graph pattern against each named graph in the dataset. For each match, the variable is bound to the IRI of the named graph where the match was found. In this example, `?g` may be bound to either `<http://example.org/graph1>` or `<http://example.org/graph2>`.

If the same IRI is used in more than one **FROM NAMED** clause, it still denotes a single named graph in the query dataset.

Combining **FROM** and **FROM NAMED**

The two clauses can be used together. Graphs specified with **FROM** form the default graph, while graphs specified with **FROM NAMED** remain distinct and are accessed with **GRAPH**.

```
SELECT ?person ?email ?source  
FROM <http://example.org/metadata>  
FROM NAMED <http://example.org/graph1>
```

```

FROM NAMED <http://example.org/graph2>
WHERE {
  ?source <http://purl.org/dc/elements/1.1/publisher> ?person .

  GRAPH ?source {
    ?x <http://xmlns.com/foaf/0.1/mbox> ?email .
  }
}

```

In this query, the first triple pattern is matched against the default graph, built from `<http://example.org/metadata>`. The **GRAPH ?source** block is matched against named graphs. The variable `?source` connects the metadata in the default graph with the named graph to be queried.

This pattern is common when the default graph contains metadata about the named graphs, such as provenance, publisher, date, or access information, while the named graphs contain the actual domain data.

The GRAPH Clause with a Fixed IRI

The **GRAPH** clause may specify a fixed graph IRI.

```

SELECT ?s ?p ?o
FROM NAMED <http://example.org/graph1>
FROM NAMED <http://example.org/graph2>
WHERE {
  GRAPH <http://example.org/graph1> {
    ?s ?p ?o .
  }
}

```

Only the named graph `<http://example.org/graph1>` is used as the active graph inside the block. Triples from `<http://example.org/graph2>` are ignored for that part of the query.

If the fixed IRI does not correspond to any named graph in the query dataset, the **GRAPH** block has no solution.

The GRAPH Clause with a Variable

When the graph name is a variable, SPARQL ranges over the named graphs in the dataset.

```

SELECT ?g ?s ?p ?o
FROM NAMED <http://example.org/graph1>
FROM NAMED <http://example.org/graph2>
WHERE {
  GRAPH ?g {
    ?s ?p ?o .
  }
}

```

The variable **?g** is bound to the name of each graph that satisfies the pattern. This form is useful when the query must discover where a match occurs, rather than restricting evaluation to one known graph.

A graph variable may also be constrained by another part of the query:

```
SELECT ?g ?date ?s ?p ?o
FROM <http://example.org/metadata>
FROM NAMED <http://example.org/graph1>
FROM NAMED <http://example.org/graph2>
WHERE {
  ?g <http://purl.org/dc/elements/1.1/date> ?date .

  GRAPH ?g {
    ?s ?p ?o .
  }
}
```

The first pattern is evaluated against the default graph and binds **?g** to graph IRIs described in the metadata. The **GRAPH ?g** block is then evaluated only against the named graphs whose IRIs are compatible with those bindings.

Summary of the Main Cases

The interaction between **FROM**, **FROM NAMED**, and **GRAPH** can be summarized as follows:

- With no **FROM** and no **FROM NAMED**, the query uses the dataset supplied by the SPARQL service or protocol request.
- With one **FROM**, ordinary graph patterns are matched against the graph specified as the default graph.
- With multiple **FROM** clauses, ordinary graph patterns are matched against the RDF merge of the specified graphs.
- With only **FROM NAMED**, the named graphs are available through **GRAPH**, while the default graph is empty.
- With both **FROM** and **FROM NAMED**, ordinary graph patterns use the default graph built from **FROM**, while **GRAPH** blocks use the named graphs declared by **FROM NAMED**.
- **GRAPH <iri>** restricts matching to the named graph identified by that IRI.
- **GRAPH ?g** ranges over the named graphs in the dataset and binds **?g** to the graph where the match is found.

1.6 Graph Patterns in the WHERE Clause

1.6.1 Basic Graph Patterns

The **WHERE** clause contains the graph pattern to be evaluated. Multiple triple patterns are separated by dots. In a basic graph pattern, all patterns must match simultaneously.

```

PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
PREFIX dbpprop: <http://dbpedia.org/property/>

SELECT ?movie ?actor ?country
FROM <http://dbpedia.org>
WHERE {
    ?movie dbpedia-owl:starring ?actor .
    ?movie dbpedia-owl:director ?actor .
    OPTIONAL { ?movie dbpprop:country ?country }
}

```

The first two triple patterns are conjunctive. A solution must bind **?movie** and **?actor** so that the same person is both starring in and directing the same movie. Reusing the same variable across patterns imposes equality of the corresponding RDF terms in the solution mapping.

1.6.2 OPTIONAL Patterns

The **OPTIONAL** keyword marks a pattern whose absence does not invalidate the solution. In the previous query, the country is retrieved when a matching country triple exists. If no such triple is present for a movie that already satisfies the mandatory patterns, the solution is still returned and **?country** remains unbound.

Optional patterns are useful when RDF data is incomplete or heterogeneous, which is common in large Linked Data collections. They allow the query to preserve core results while enriching them with additional information when available.

1.6.3 UNION Patterns

The **UNION** keyword expresses alternatives between groups of graph patterns. It corresponds to a logical disjunction between pattern groups.

```

PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
PREFIX dbpprop: <http://dbpedia.org/property/>

SELECT ?movie ?person ?country
FROM <http://dbpedia.org>
WHERE {
    { ?movie dbpedia-owl:starring ?person }
    UNION
    { ?movie dbpedia-owl:director ?person }

    OPTIONAL { ?movie dbpprop:country ?country }
}

```

This query returns people related to a movie either as actors or as directors. The alternatives are enclosed in braces because each side of **UNION** is a graph pattern. After a solution is produced by either branch, the optional country pattern is evaluated in the same way as for any other solution.

Basic conjunctions, optional patterns, and unions can be nested to express complex graph constraints. Care is required when variables appear in only one branch of a union, since they may remain unbound in solutions produced by the other branch.

1.6.4 FILTER Conditions

A **FILTER** restricts the solutions produced by graph pattern matching. It is evaluated as a boolean expression over the current solution bindings.

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
PREFIX dbpprop: <http://dbpedia.org/property/>
```

```
SELECT DISTINCT ?actor
FROM <http://dbpedia.org>
WHERE {
    ?movie dbpedia-owl:starring ?actor .
    ?movie dbpprop:budget ?budget .
    FILTER(isNumeric(?budget) && ?budget > 1.0E8)
}
```

The query retrieves actors starring in movies whose budget is numeric and greater than 100 million. The **DISTINCT** modifier removes duplicate actor bindings, which can arise when the same actor appears in several matching movies.

The expression `isNumeric(?budget)` guards the numeric comparison. Without such a check, heterogeneous literal datatypes in the dataset may produce evaluation errors or unexpected filtering behavior. More generally, **FILTER** is the main mechanism for applying value constraints after structural graph matching has produced candidate solutions.

SPARQL also supports existence tests in filters. **FILTER EXISTS** keeps a solution only if an additional pattern can be matched, whereas **FILTER NOT EXISTS** keeps it only if the pattern cannot be matched. These constructs are useful when the inclusion or exclusion condition is itself graph-shaped rather than a simple comparison.

1.6.5 GRAPH Patterns

The **GRAPH** keyword evaluates a pattern against a named graph instead of the default graph. The graph name may be an explicit IRI or a variable.

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
```

```
SELECT ?super ?sub
FROM <http://dbpedia.org>
WHERE {
    GRAPH <http://www.w3.org/2002/07/owl#> {
        ?super a rdfs:Class .
        ?sub rdfs:subClassOf ?super
    }
}
```


This query matches the class and subclass pattern inside the named graph identified by the IRI in the **GRAPH** clause. The default graph declared by **FROM** is not used for the enclosed pattern.

A graph name can also be bound through a variable:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?super ?sub
FROM NAMED <http://www.w3.org/2002/07/owl#>
FROM NAMED <http://www.openlinksw.com/schemas/oplweb#>
WHERE {
  GRAPH ?src {
    ?super a rdfs:Class .
    ?sub rdfs:subClassOf ?super
  }
}
```

Here, **?src** ranges over the named graphs declared by **FROM NAMED**. The variable does not need to be projected in the result. It can serve only as an intermediate binding that selects the graph in which the enclosed pattern is evaluated.

1.7 Query Modifiers

1.7.1 Ordering

ORDER BY orders the solution sequence according to one or more expressions, typically variables.

ORDER BY ?budget

The default order is ascending. Descending order is requested with **DESC**:

ORDER BY DESC(?budget)

Ordering is applied to the solution sequence after pattern matching and filtering. It is therefore a result-set operation, not a graph-matching condition.

1.7.2 Limiting and Offsetting Results

LIMIT sets the maximum number of solutions returned. **OFFSET** skips a fixed number of solutions from the beginning of the current solution sequence. Their typical use is pagination or sampling of large result sets.

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
PREFIX dbpprop: <http://dbpedia.org/property/>

SELECT ?actor ?movie ?budget
FROM <http://dbpedia.org>
WHERE {
  ?movie dbpedia-owl:starring ?actor .
  ?movie dbpprop:budget ?budget .
}
```

```
    FILTER(isNumeric(?budget) && ?budget > 1.0E8)
  }
ORDER BY ?budget
LIMIT 100
OFFSET 10
```

This query returns at most 100 solutions after skipping the first 10 solutions in the sequence ordered by increasing budget. Since **OFFSET** is meaningful only relative to an order, pagination queries should normally include an explicit **ORDER BY** when deterministic behavior is required.

1.8 Selected Operators and Functions

SPARQL expressions can be used in filters and in other expression positions. The operators and functions in tables 1.1 and 1.2 are commonly used for inspecting RDF terms, checking variable bindings, manipulating string literals, and constraining query results.

The term-inspection functions are particularly useful when querying heterogeneous RDF data, where the same property may be used with IRIs, blank nodes, literals, or literals having different datatypes. For example, **isIRI** distinguishes resources from literal values, **isBlank** detects anonymous resources, and **datatype** allows filters to depend on the datatype of a literal. String operators such as **contains** and **regex** support textual constraints, while **bound** is commonly used to test whether an optional pattern has contributed a value to the current solution.

1.9 Query Forms

1.9.1 SELECT

SELECT returns the bindings of the variables named in the projection. It is the most common query form when the expected result is a table-like result set.

```
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
```

```
SELECT ?movie ?actor
WHERE {
  ?movie dbpedia-owl:starring ?actor
}
```

Only projected variables appear in the result, even if additional variables are used internally for matching, filtering, or graph selection.

Unary operator	Argument type	Return type	Meaning
bound(A)	variable	xsd:boolean	Returns true if the variable A has a value in the current solution. It is especially useful with OPTIONAL patterns.
isIRI(A) or isURI(A)	RDF term	xsd:boolean	Tests whether A is an IRI. The two names are equivalent; isIRI is the preferred terminology.
isBlank(A)	RDF term	xsd:boolean	Tests whether A is a blank node.
isLiteral(A)	RDF term	xsd:boolean	Tests whether A is a literal, including plain literals, language-tagged strings, and typed literals.
isNumeric(A)	RDF term	xsd:boolean	Tests whether A is a numeric literal, so that numeric comparisons and arithmetic operations can be applied safely.
ucase(A)	string literal	string literal	Returns the uppercase form of the string literal A .
datatype(A)	literal	IRI	Returns the datatype IRI of the literal A . For example, it can be used to distinguish xsd:integer , xsd:decimal , and xsd:date literals.

Table 1.1: SPARQL unary operators

Binary operator	Type of A	Type of B	Return type	Meaning
contains(A,B)	string literal	string literal	xsd:boolean	Returns true if the string literal A contains the string literal B .
regex(A,B)	string literal	simple literal	xsd:boolean	Returns true if the string literal A matches the regular expression pattern B .

Table 1.2: SPARQL binary operators

1.9.2 CONSTRUCT

CONSTRUCT returns an RDF graph generated from a template. The query first evaluates the **WHERE** pattern, then instantiates the construct template for each solution.

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>
PREFIX dbpprop: <http://dbpedia.org/property/>
PREFIX vcard: <http://www.w3.org/2006/vcard/ns#>
```

```
CONSTRUCT {
  ?actor vcard:birthday ?birthdate
}
FROM <http://dbpedia.org>
WHERE {
  dbpedia:The_Matrix_Revolutions dbpedia-owl:starring ?actor .
  ?actor dbpprop:birthDate ?birthdate
}
```

The query retrieves actors starring in **dbpedia:The_Matrix_Revolutions** together with their birth dates. For each solution, it creates a triple using the **vcard:birthday** predicate. If ten actors with birth dates match the pattern, the constructed result graph contains ten corresponding triples, subject to RDF graph semantics such as duplicate triple elimination.

CONSTRUCT is useful when the goal is not merely to inspect RDF data, but to transform it into another RDF shape, possibly using a different vocabulary.

1.9.3 ASK

ASK checks whether a graph pattern has at least one solution. It returns only a boolean value.

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>

ASK {
  dbpedia:The_Matrix dbpedia-owl:starring dbpedia:Keanu_Reeves
}
```

The result is **true** if the triple pattern is matched in the queried dataset, and **false** otherwise. Variables may also appear in **ASK** patterns. In that case, the answer is **true** if there exists at least one binding that satisfies the pattern.

1.9.4 DESCRIBE

DESCRIBE returns an informative RDF graph about one or more resources. Unlike **CONSTRUCT**, the exact shape of the returned graph is not specified by an explicit template in the query.

```
PREFIX dbpedia: <http://dbpedia.org/resource/>
PREFIX dbpedia-owl: <http://dbpedia.org/ontology/>

DESCRIBE ?actor {
  dbpedia:The_Matrix_Revolutions dbpedia-owl:starring ?actor
}
```

This query first identifies actors starring in **dbpedia:The_Matrix_Revolutions**, then asks the endpoint to return descriptive RDF data about those resources. The amount and selection of returned information depend on the endpoint. A typical implementation may include triples where the described resource appears as subject or object, but the language intentionally leaves this behavior implementation-dependent.

1.10 Scope of the Query Language

The query forms **SELECT**, **CONSTRUCT**, **ASK**, and **DESCRIBE** cover the principal ways of extracting information from RDF datasets: tabular bindings, generated RDF graphs, boolean existence checks, and descriptive graphs. The query language described here is concerned with retrieval and graph construction as query results. Modification of RDF graphs is handled by separate update mechanisms rather than by these query forms.

The essential modelling idea remains the same across the language: RDF data is queried by matching graph-shaped patterns, binding variables to RDF terms, and then projecting, filtering, ordering, or reshaping the resulting solution mappings.